T

TPOS

THE OPERATING SYSTEM FOR TRAILER DEALERS

Product Specification

A business-level, implementation-agnostic specification — sufficient for an independent team to rebuild the system from scratch in the stack and architecture of their choosing.

Document v1.0 · Functional & non-functional requirements · B&W wireframes · Confidential draft

Contents

1. Product Vision & Business Model
2. Multi-Tenancy & Deployment Modes
3. Personas & Roles
4. Domain Model — One Record, Many Lenses
5. The Trailer Lifecycle
6. Functional Specification by Role
7. Public Storefront
8. Cross-Cutting Capabilities
9. Reporting & KPIs
10. Roles & Permissions Matrix
11. External Integrations
12. Non-Functional Requirements
13. Acceptance Criteria
14. Glossary
15. Appendix A — Data Dictionary & Relational Schema
16. Appendix B — API Reference
17. Appendix C — Performance & Scaling
18. Appendix D — Architecture Decisions & Roads Not Taken

1 · Product Vision & Business Model

TPOS is a vertical SaaS that gives an independent trailer dealership one system to run both its **public storefront** and its entire **back office** — acquisition, reconditioning, pricing, marketing, sales, financing, titling, delivery and bookkeeping — on a single shared record of truth.

The market is fragmented small-to-mid dealers (used & refurbished commercial trailers: dry vans, flatbeds, reefers, lowboys, dumps, tankers, chassis) who today stitch together a website builder, a generic CRM, spreadsheets, QuickBooks and listing sites (TruckPaper, Facebook). TPOS replaces that stack with a purpose-built, role-aware platform.

Revenue model

Offering	Pricing	For	Notes
Hosted — Starter	$199 / month	Single lot, ≤ ~25 units	Storefront + CRM + inventory; core roles
Hosted — Growth	$499 / month	Multi-lot dealers	All roles, financing, dispatch, reporting
Hosted — Enterprise	$1,290 / month	Large / multi-location	Unlimited units & seats, API, SSO, priority support
Annual License	$9,900 / year	Dealers wanting to self-host	Right to run the software on own infrastructure; updates included
One-Time Purchase	$24,000 + $3,000/yr support	Buy-to-own	Perpetual self-host license; optional support & updates

The platform operator (TPOS, Inc.) additionally runs a **super-admin control plane** to provision tenants, meter usage, bill subscriptions, and support dealers (see §6.12).

Product principles

- **One record, many lenses.** A trailer is simultaneously inventory, a deal, a work order, an invoice line, a title file, a delivery, and a public listing. Every role edits the *same* entity through a view shaped to its job — never a copy.
- **The storefront is generated, not separate.** Inventory marked *available* is automatically published to the dealer's branded public site and syndicated to listing channels.
- **Role-first, not module-first.** Each user lands on a workspace built around their daily goals and decisions, not a generic menu of tables.

- **Auditable by default.** Price changes, discounts, status transitions and money movement are logged with who/when/why.

2 · Multi-Tenancy & Deployment Modes

Each dealership is a **tenant** with isolated data, its own users, branding, lots, inventory and storefront domain. The system must support three deployment modes from one codebase:

Mode	Hosting	Tenancy	Billing
Hosted (SaaS)	Operator-run cloud	Many tenants per deployment (logical isolation, per-tenant scoping on every query)	Recurring subscription, usage-metered
Licensed self-host	Dealer infrastructure	Typically single tenant; multi-tenant allowed	Annual license key, update channel
One-time self-host	Dealer infrastructure	Single tenant	Perpetual key; optional support plan

Requirement. Tenant isolation must be enforced at the data layer (no cross-tenant read/write possible), branding/config is per-tenant, and a tenant's storefront resolves on a custom domain or subdomain. The control plane (§6.12) exists only in hosted mode.

3 · Personas & Roles

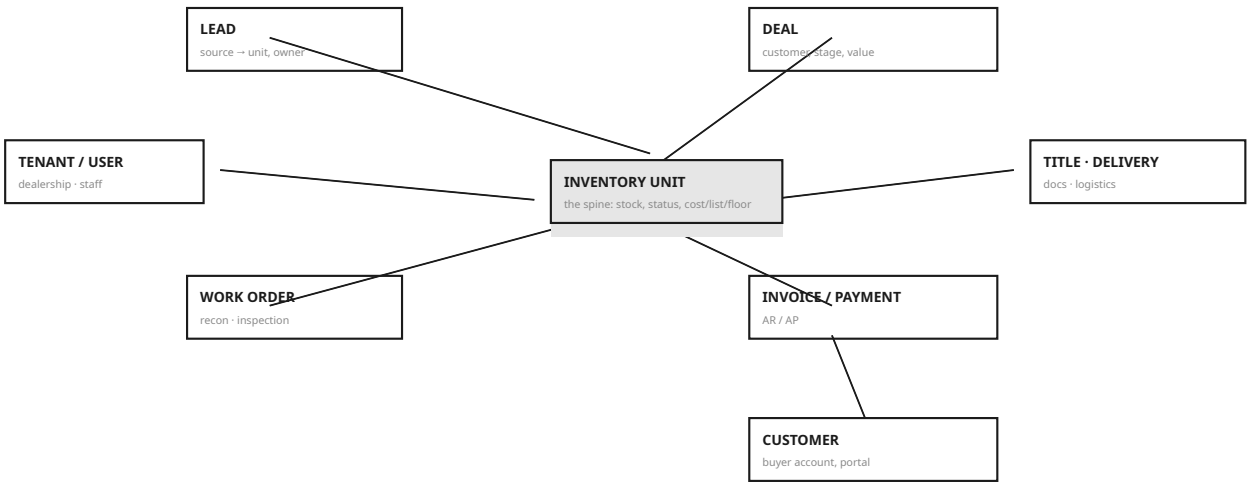
TPOS serves twelve roles across the dealership, plus the dealership's own customers and the platform operator. Each persona has a primary goal and a small set of "jobs to be done" that the system must make fast.

Role	Primary goal	Key jobs-to-be-done
Owner / Dealer Principal	Run a healthy, profitable business	See revenue, gross, turn & pipeline at a glance; spot aged inventory, overdue AR, stalled deals; manage the team
Lead Generation / BDC	Capture & route every inquiry fast	Triage the lead inbox, measure response time & sources, run campaigns, assign to sales
Salesperson	Work the pipeline & close	Advance deals through stages, build quotes, track follow-ups, manage customers
Sales Manager	Coach, forecast, approve	Compare rep performance, forecast the month, approve below-floor discounts
Pricing Analyst	Maximize margin & turn	Reprice against age & demand, monitor margin, compare market comps, approve changes

Inventory Manager	Source, recon, place & turn units	Track stock by status & location, manage intake & aging, push units into recon & to the lot
Service Technician	Recon & inspect units quickly	Work the work-order board, complete inspection checklists, log hours & parts
Office / Titles & Admin	Keep paperwork clean & legal	Track titles/DMV, collect deal documents, schedule signings & deliveries
Bookkeeper / Accounting	AR/AP, cash, month-end	Issue & collect invoices, record payments, pay bills, close the month, report P&L
Delivery Dispatch	Move units to buyers	Schedule deliveries, assign drivers/carriers, quote freight, track in-transit
Customer (buyer)	Buy with confidence, self-serve	View quotes, apply for financing, track delivery, sign & download documents
Platform Super-Admin	Operate the SaaS business	Provision & bill tenants, watch MRR/churn, monitor usage & system health, support dealers

4 · Domain Model — One Record, Many Lenses

The data model is small and tightly linked. The **Trailer (Inventory Unit)** is the spine; everything references it.



Entity-relationship overview. One trailer threads through inventory, deals, work orders, invoices, title and delivery; each role reads the same graph.

Core entities

Entity	Purpose	Key attributes
Tenant	A dealership	name, plan, branding, lots[], storefront domain, billing
User	A staff member	name, role(s), permissions, lot scope
Inventory Unit (Trailer)	The spine	stock #, type, year, make, length, axles, capacity, condition, photos, **cost**, **list**, **floor**, status, location, acquired/sold dates, views, lead-count
Lead	An inbound prospect	name, company, source, interest→Unit, status, owner→User, value, created
Customer	A buyer account	name, company, contacts, deals[], documents[]
Deal	A sales opportunity	customer, unit, stage, value, probability, salesperson, financing
Quote	A priced offer	unit, line items (price, prep, freight, tax), total, validity
Work Order	Recon / inspection / repair	unit, type, status, technician, tasks[], parts cost, hours est/actual

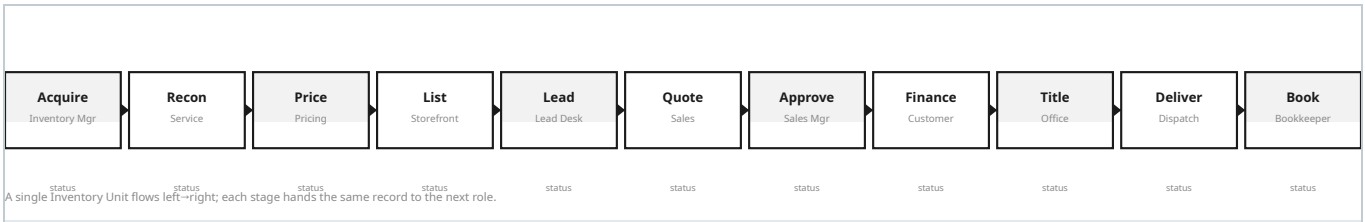
Invoice / Payment	Money in & out (AR/AP)	customer/vendor, unit, amount, status, dates, method
Title / Document	Legal & DMV paperwork	unit, title status, state, lien, document checklist & signatures
Delivery	Logistics of handoff	unit, deal, origin, destination, driver/carrier, status, ETA, freight cost

The status machine

An Inventory Unit moves through: `incoming → recon → available → pending → sold`. Status is the single field that re-shapes the unit across roles — e.g. *available* publishes it to the storefront; *pending* hides it from new leads; *sold* moves it to delivery & books revenue. The system must guard valid transitions and log each one.

5 · The Trailer Lifecycle

The end-to-end flow ties every role together. Each stage hands the same unit to the next role.



Trailer lifecycle — the same unit handed role to role from acquisition through booking the sale.

6 · Functional Specification by Role

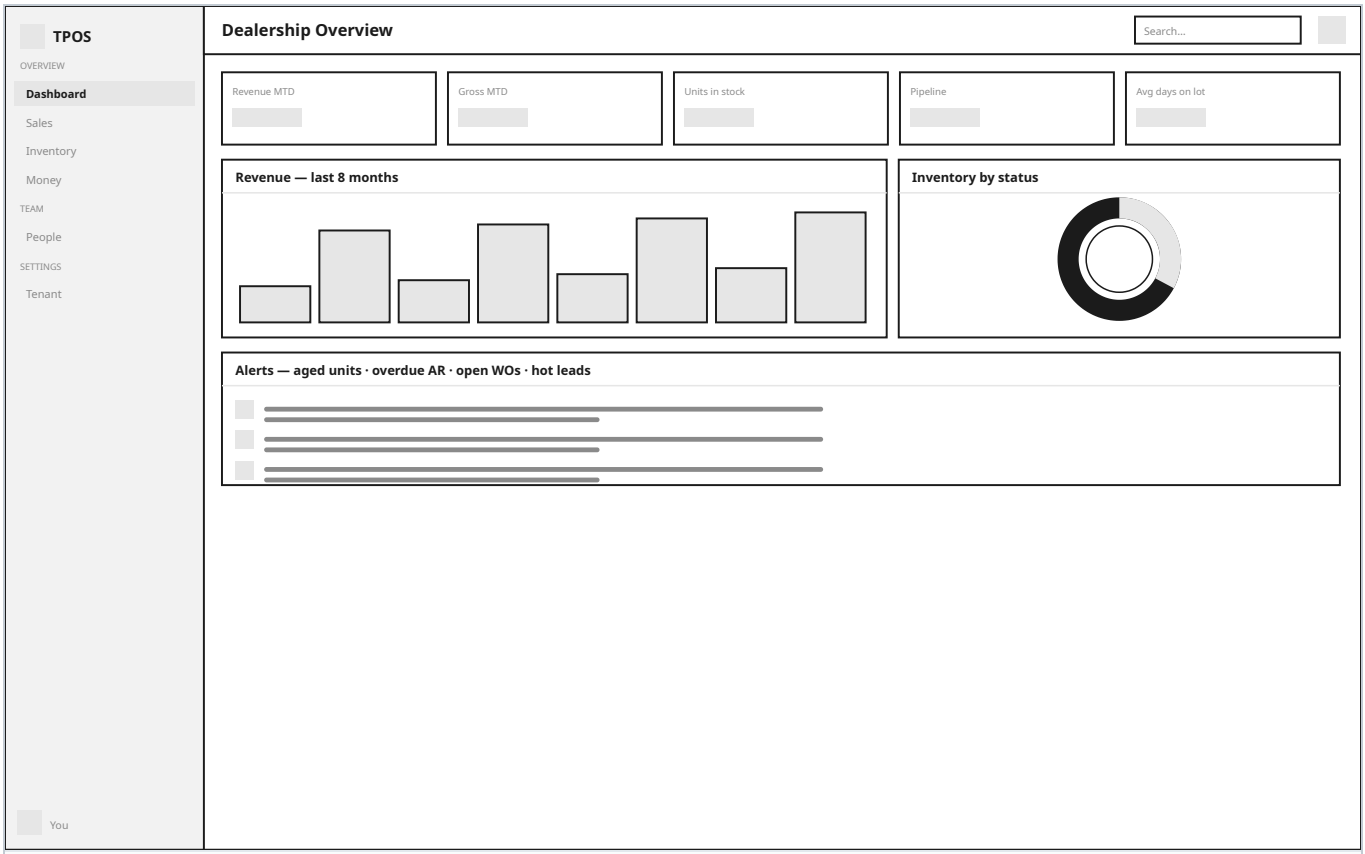
Each role below states its **goal**, the **screens** it owns, its **functional requirements** (what the system must let it do), and a **wireframe** of the primary workspace. Wireframes define information architecture and layout only — visual design, components and stack are the build team's choice.

6.1 Owner / Dealer Principal

Goal. Run a healthy, profitable business at a glance.

Functional requirements

- Executive dashboard: revenue, gross, units, pipeline, days-on-lot.
- Cross-role alert feed: aged inventory, overdue AR, open work orders, hot/new leads.
- Drill into Sales and Inventory; manage team & tenant settings & billing.



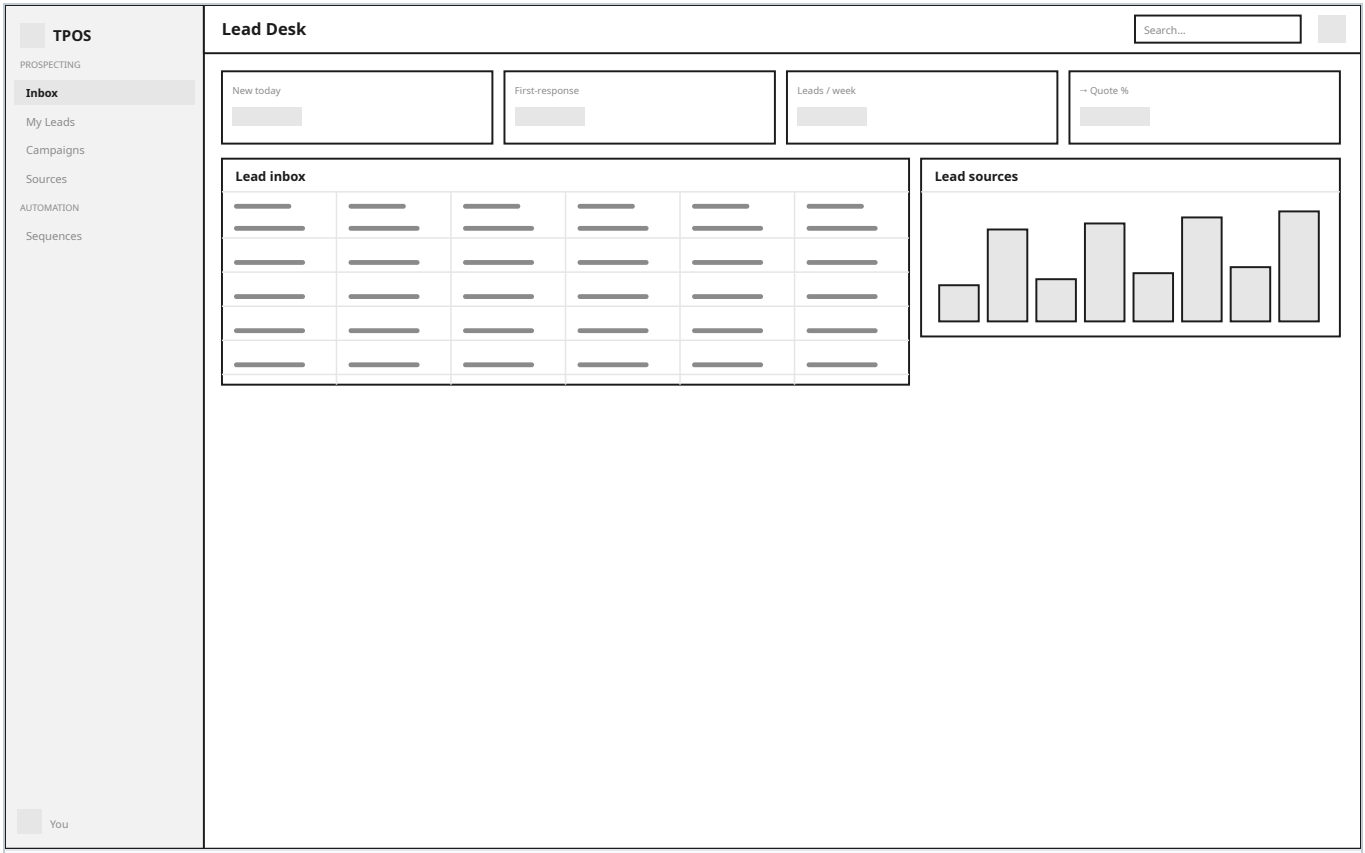
Wireframe — Owner / Dealer Principal primary workspace (layout & IA only).

6.2 Lead Generation / BDC

Goal. Capture every inquiry and route it fast.

Functional requirements

- Unified lead inbox (web, listing sites, phone, walk-in) with age & first-response timers.
- Assign/route leads to salespeople; one-click call/email.
- Source & campaign analytics: volume, cost-per-lead, conversion to quote.



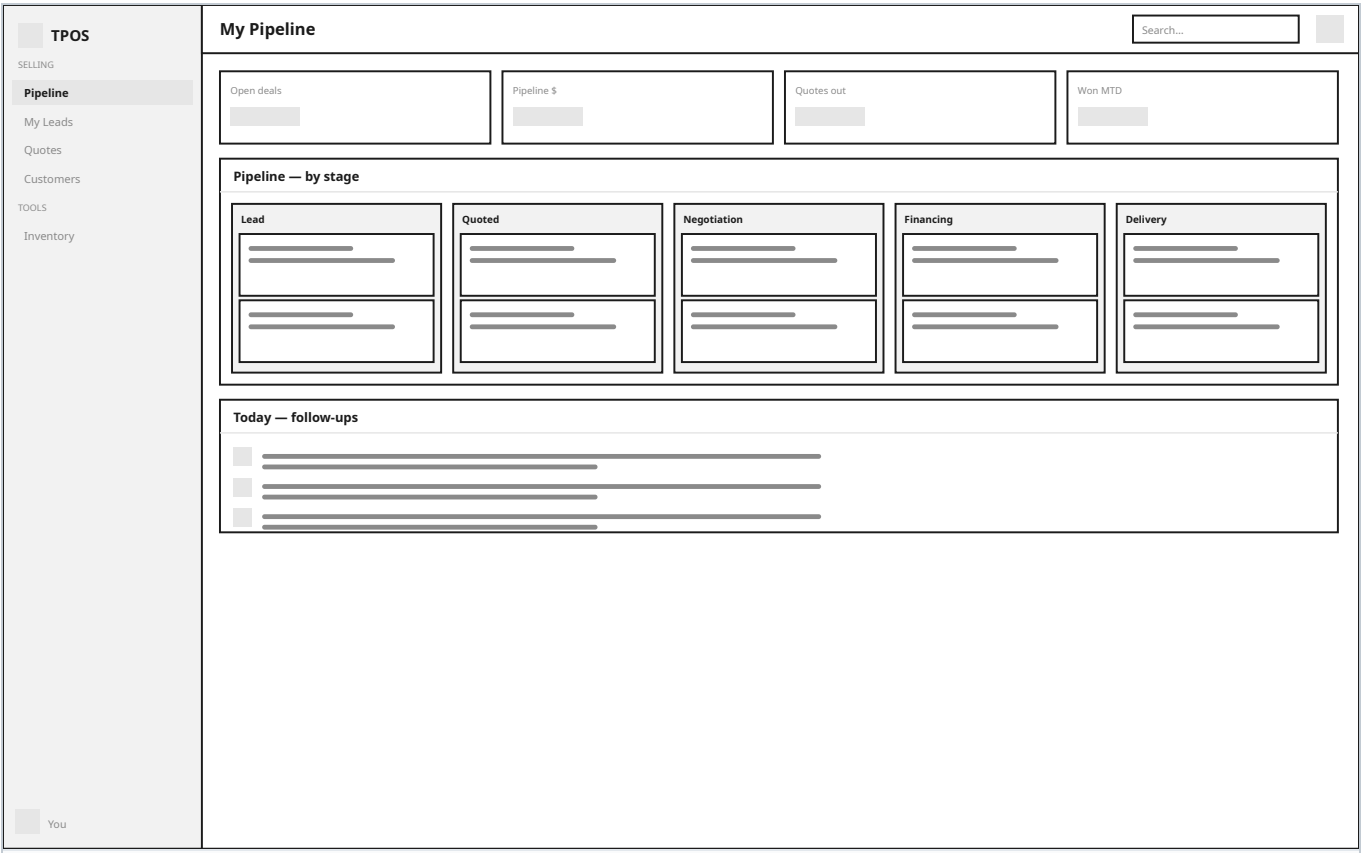
Wireframe — Lead Generation / BDC primary workspace (layout & IA only).

6.3 Salesperson

Goal. Work the pipeline and close deals.

Functional requirements

- Kanban pipeline of own deals across stages with probability & last-touch.
- Quote builder tied to a unit (price, prep, freight, tax → total).
- Daily follow-up tasks; customer accounts and history.



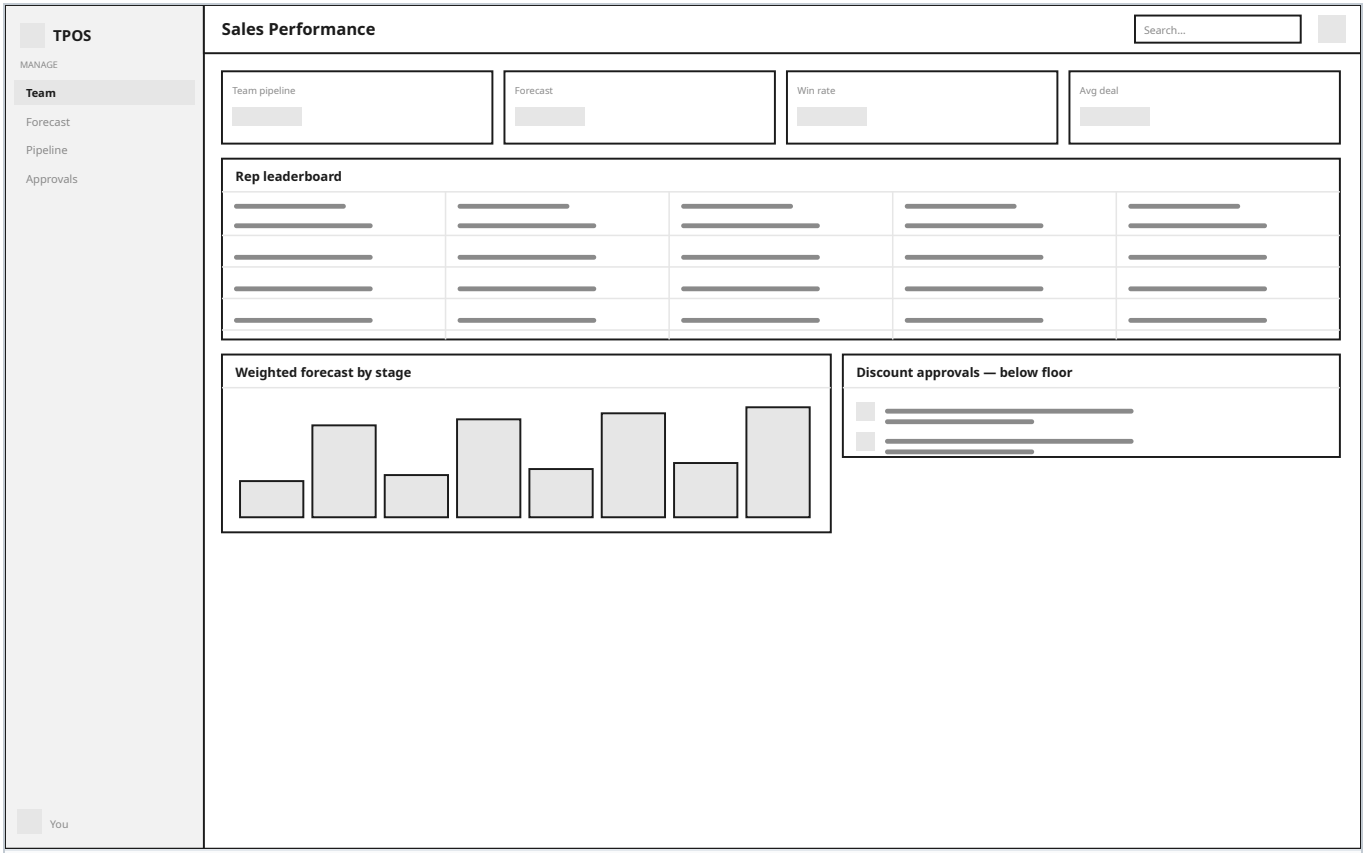
Wireframe — Salesperson primary workspace (layout & IA only).

6.4 Sales Manager

Goal. Coach the team, forecast, approve.

Functional requirements

- Rep leaderboard (deals, pipeline, won, win-rate) and conversion funnel.
- Weighted forecast by stage.
- Approve/deny/counter below-floor discounts — logged with reason.



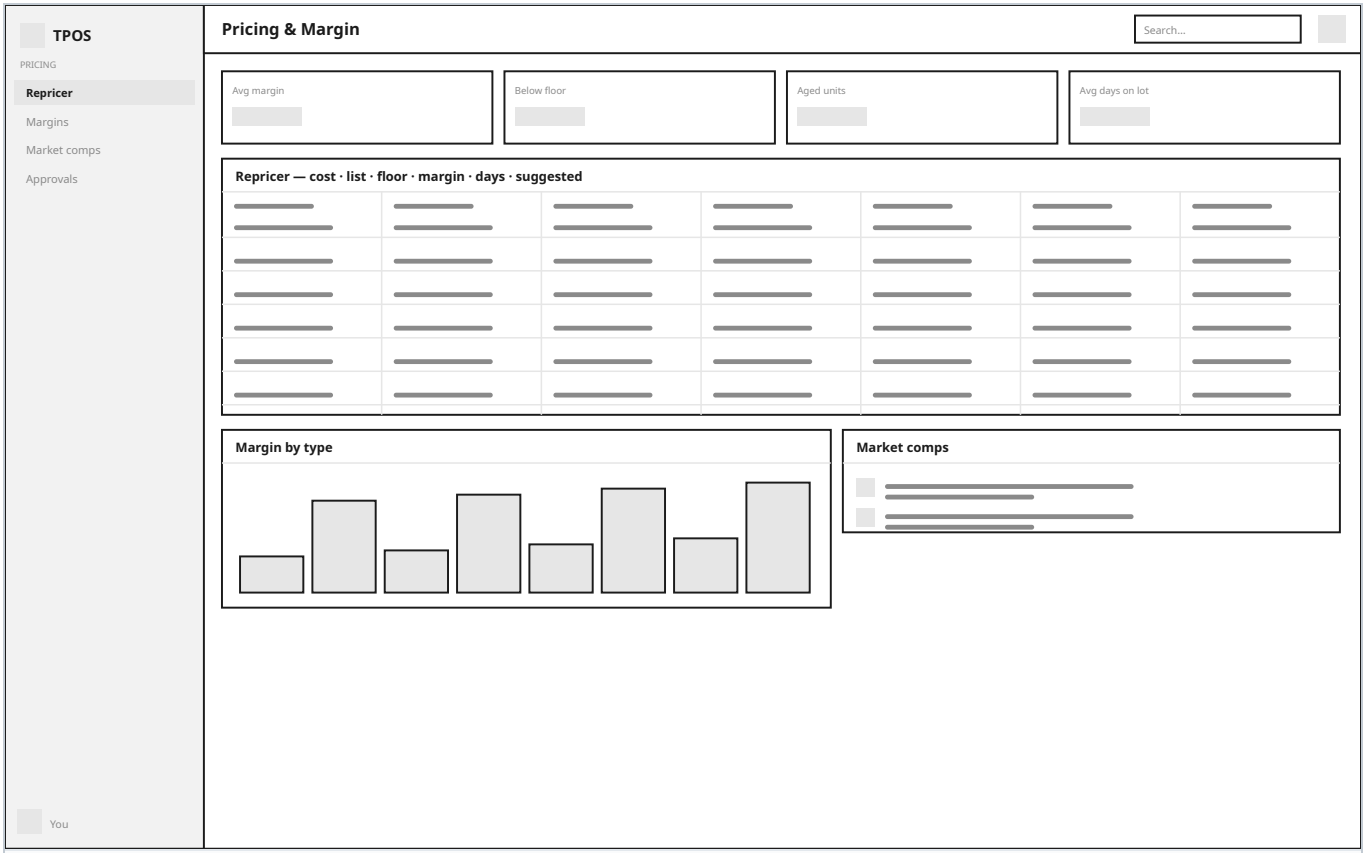
Wireframe — Sales Manager primary workspace (layout & IA only).

6.5 Pricing Analyst

Goal. Maximize margin and turn.

Functional requirements

- Repricer over available units: cost/list/floor/margin/days/demand with a suggested price & reason.
- Margin monitoring by type; aged-unit reprice prompts.
- Market comparables; price-change approval queue.



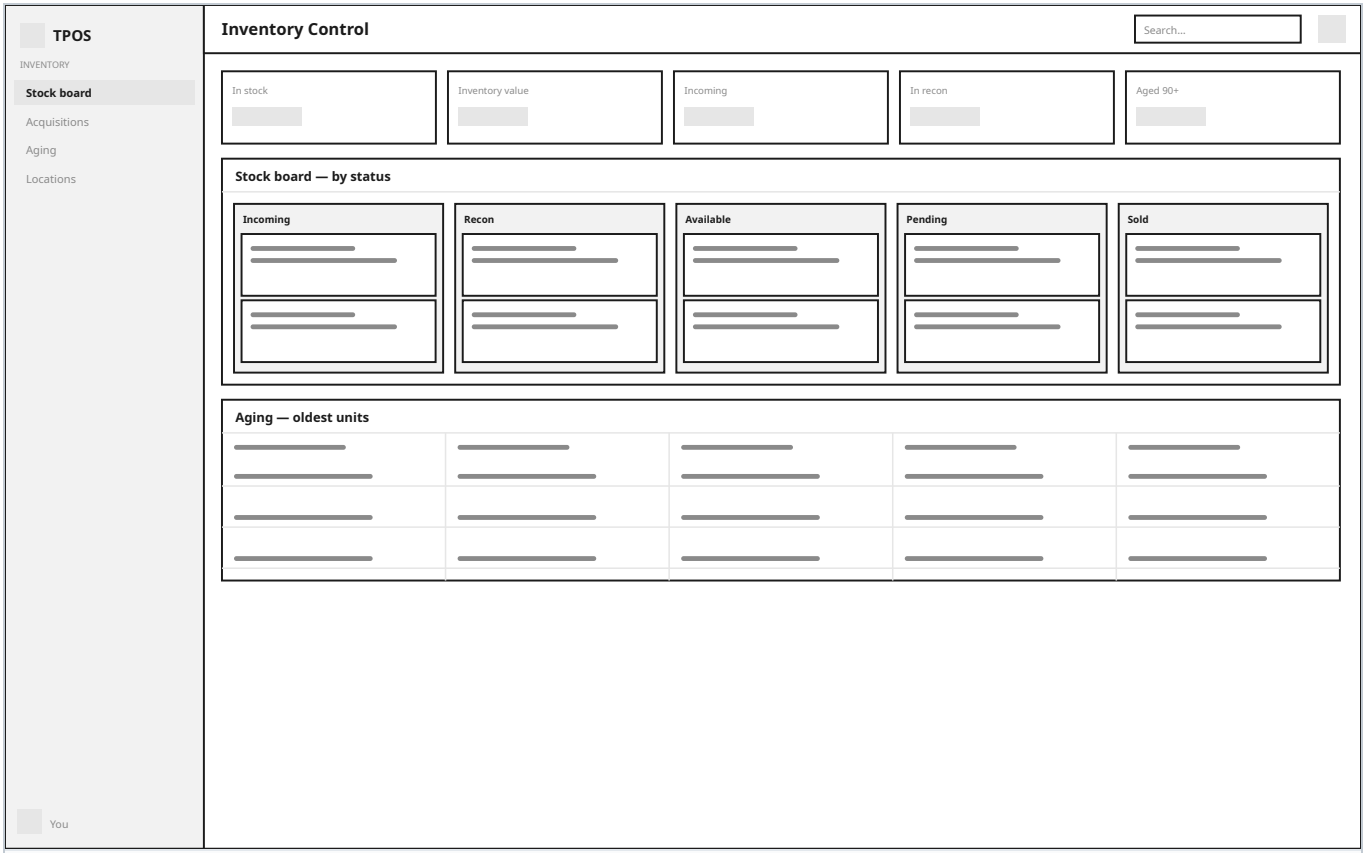
Wireframe — Pricing Analyst primary workspace (layout & IA only).

6.6 Inventory Manager

Goal. Source, recon, place and turn units.

Functional requirements

- Stock board by status (incoming→recon→available→pending→sold) and by location/lot.
- Acquisition intake with ETA & cost; push units into recon and onto the lot.
- Aging report and turn metrics; trigger reprice/clearance.



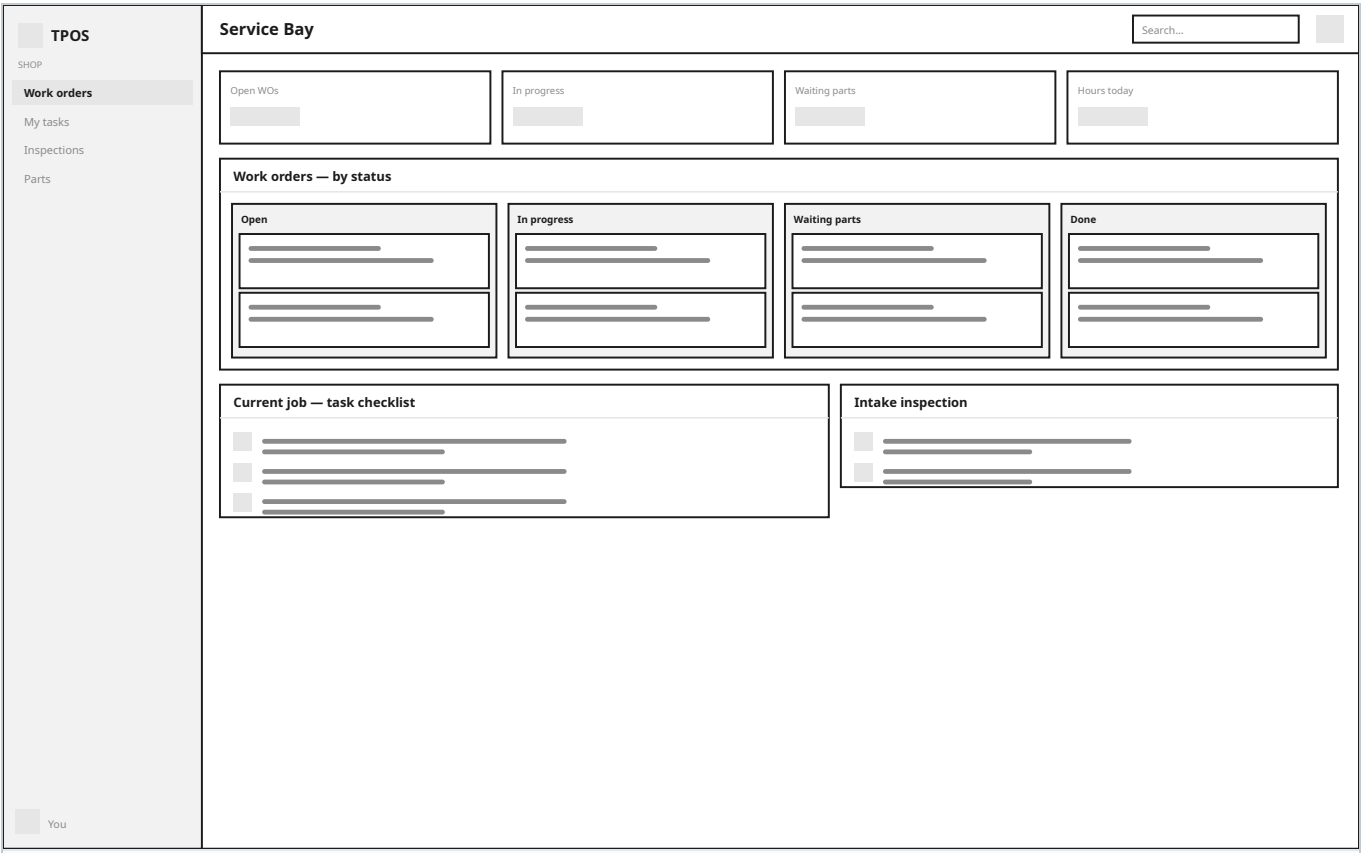
Wireframe — Inventory Manager primary workspace (layout & IA only).

6.7 Service Technician

Goal. Recondition and inspect units quickly.

Functional requirements

- Work-order board by status with hours-vs-estimate and parts cost.
- Per-job task checklist; intake inspection checklists (brakes/tires/lights/floor).
- Log hours, parts and photos against the unit.



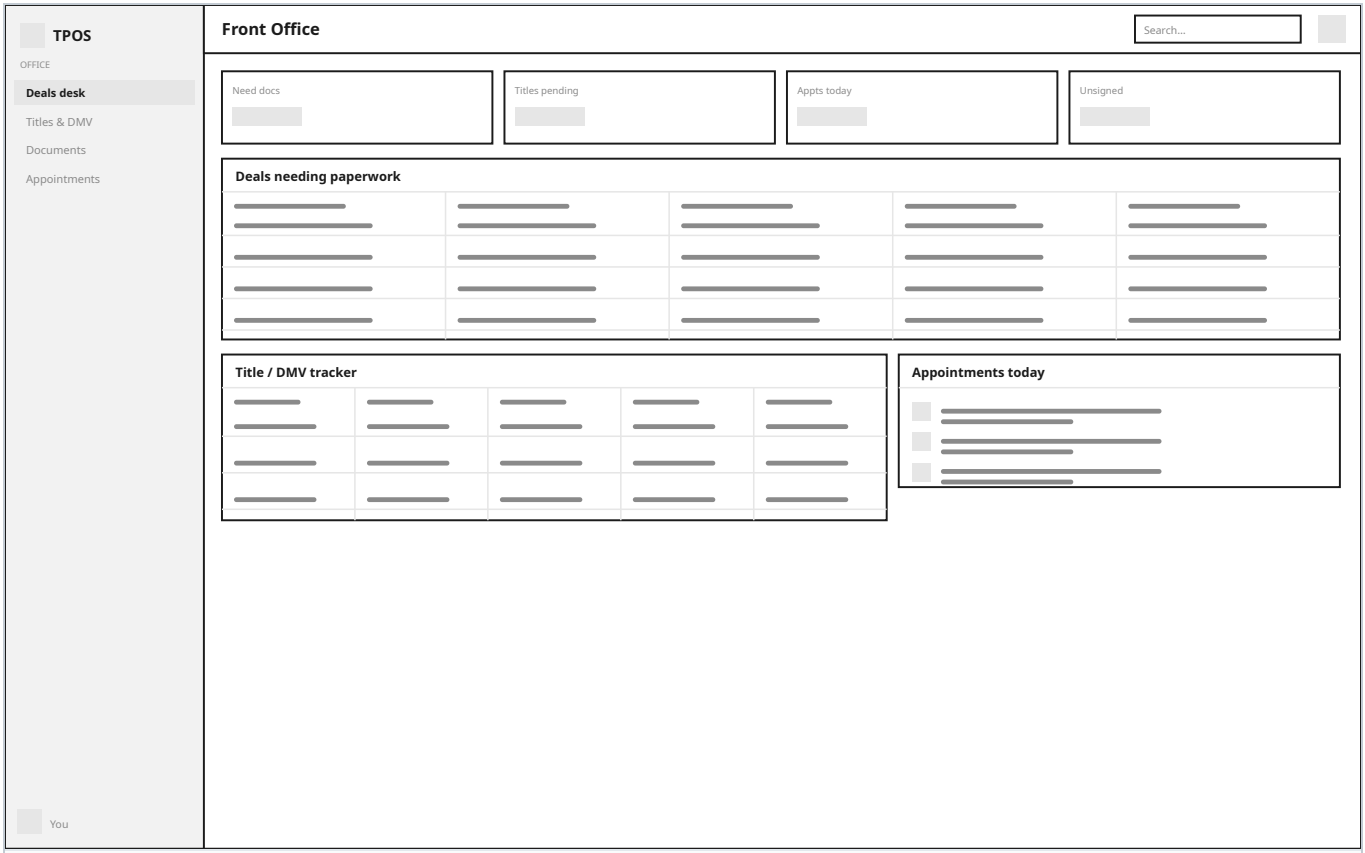
Wireframe — Service Technician primary workspace (layout & IA only).

6.8 Office / Titles & Admin

Goal. Keep paperwork clean and legal.

Functional requirements

- Deals-needing-documents queue with missing-doc flags (title, bill of sale, financing, insurance).
- Title / DMV tracker by unit (in hand / applied / pending / lien release) per state.
- Appointments & signings; document checklist & e-signature.



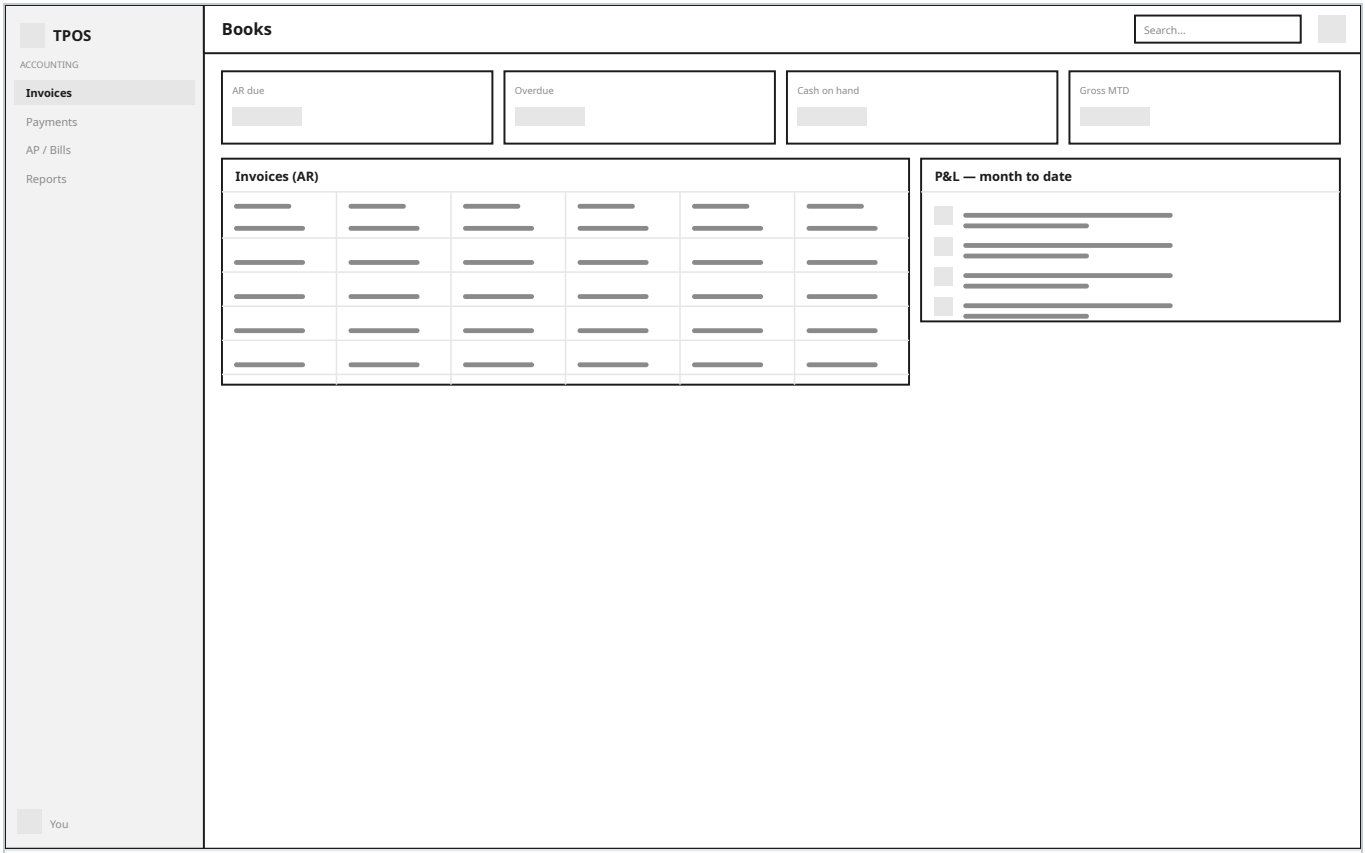
Wireframe — Office / Titles & Admin primary workspace (layout & IA only).

6.9 Bookkeeper / Accounting

Goal. AR/AP, cash and month-end.

Functional requirements

- Invoices (AR) with status (paid/due/overdue) and record-payment.
- AP / vendor bills; cash position; payments ledger.
- Month-end P&L (revenue, COGS, gross, expenses, net); accounting export.



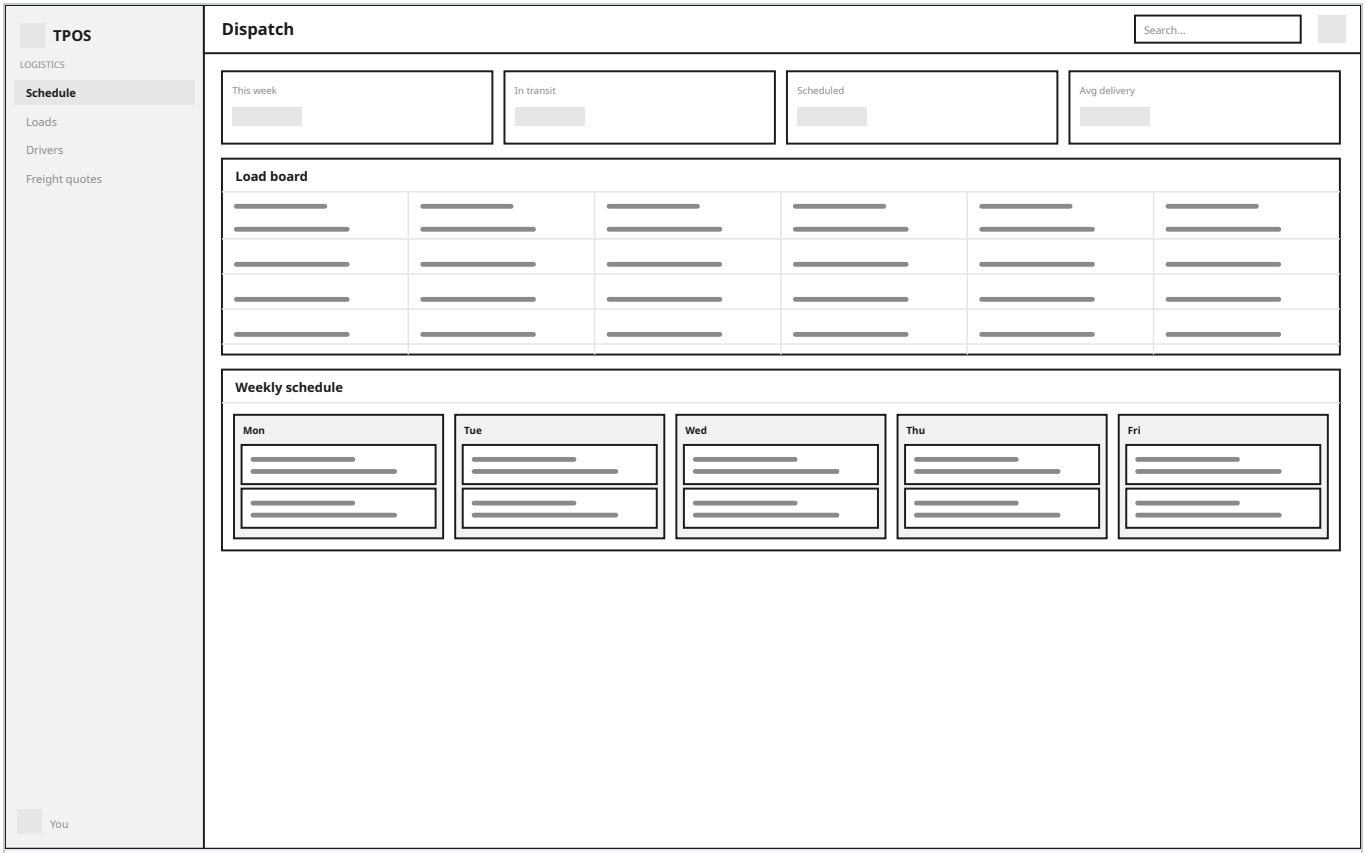
Wireframe — Bookkeeper / Accounting primary workspace (layout & IA only).

6.10 Delivery Dispatch

Goal. Move sold units to buyers.

Functional requirements

- Load board of units needing delivery with destination, carrier/driver and status.
- Weekly delivery schedule; assign drivers/carriers.
- Freight rate quotes by lane; in-transit tracking with ETA.



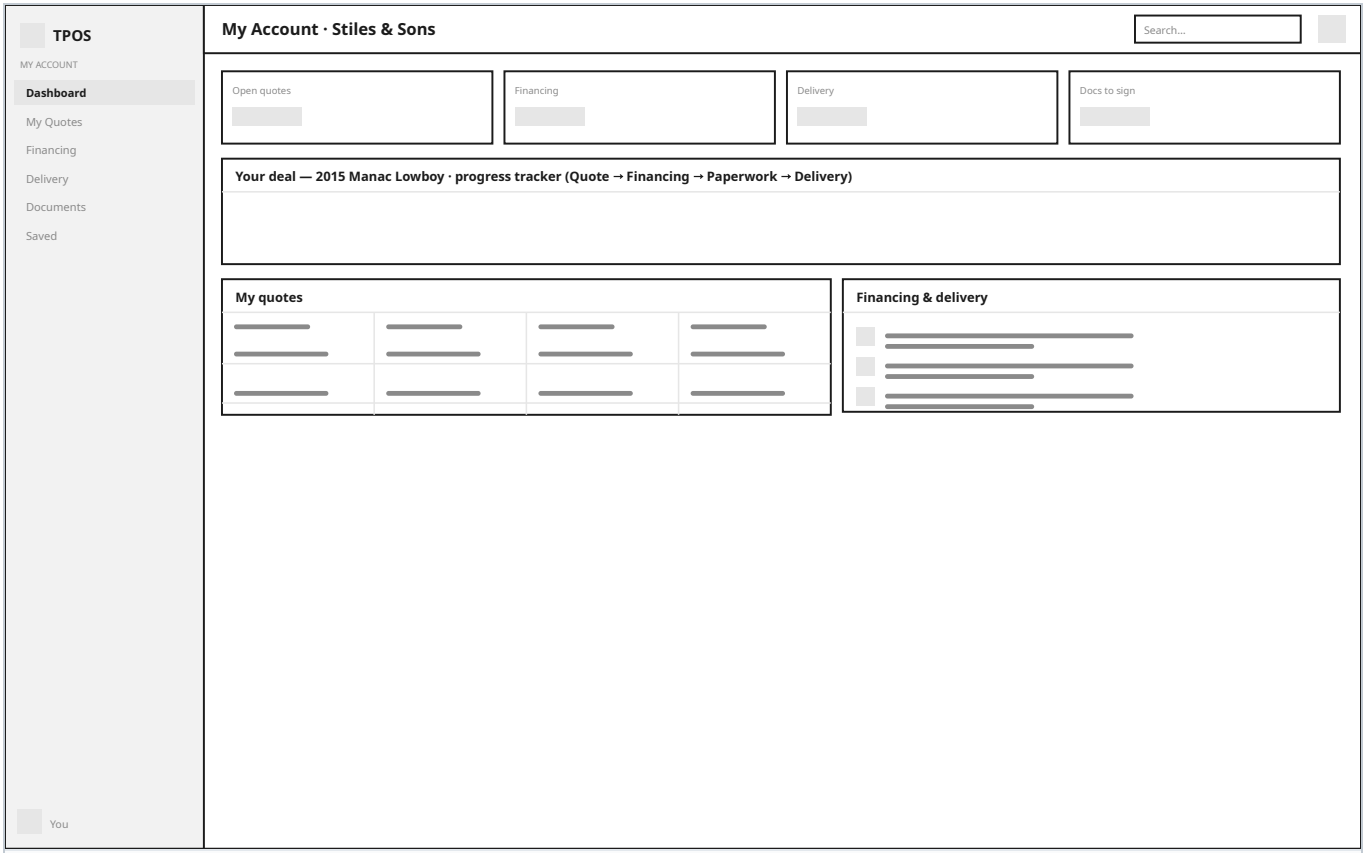
Wireframe — Delivery Dispatch primary workspace (layout & IA only).

6.11 Customer (Buyer) Portal

Goal. Buy with confidence, self-serve.

Functional requirements

- Deal progress tracker (quote → financing → paperwork → delivery).
- View/accept quotes; apply for financing and see decision status.
- Track delivery; review, e-sign and download documents; saved trailers.



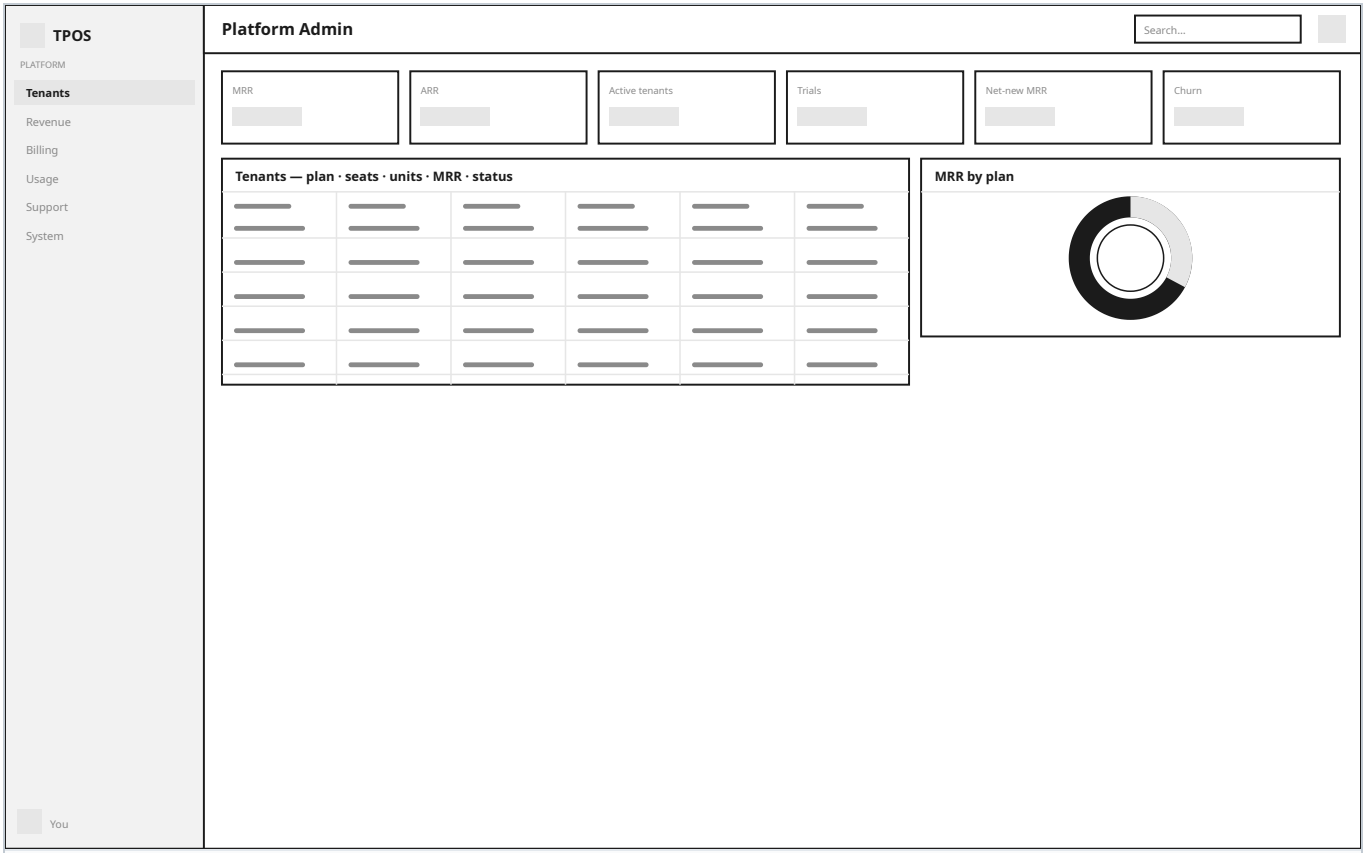
Wireframe — Customer (Buyer) Portal primary workspace (layout & IA only).

6.12 Platform Super-Admin

Goal. Operate the SaaS business (hosted mode).

Functional requirements

- Tenant directory with plan, seats, units, MRR and status; provision & manage.
- Revenue (MRR/ARR/net-new/churn), MRR-by-plan, trials & onboarding.
- Cross-tenant usage metering, support queue and system-health monitoring.



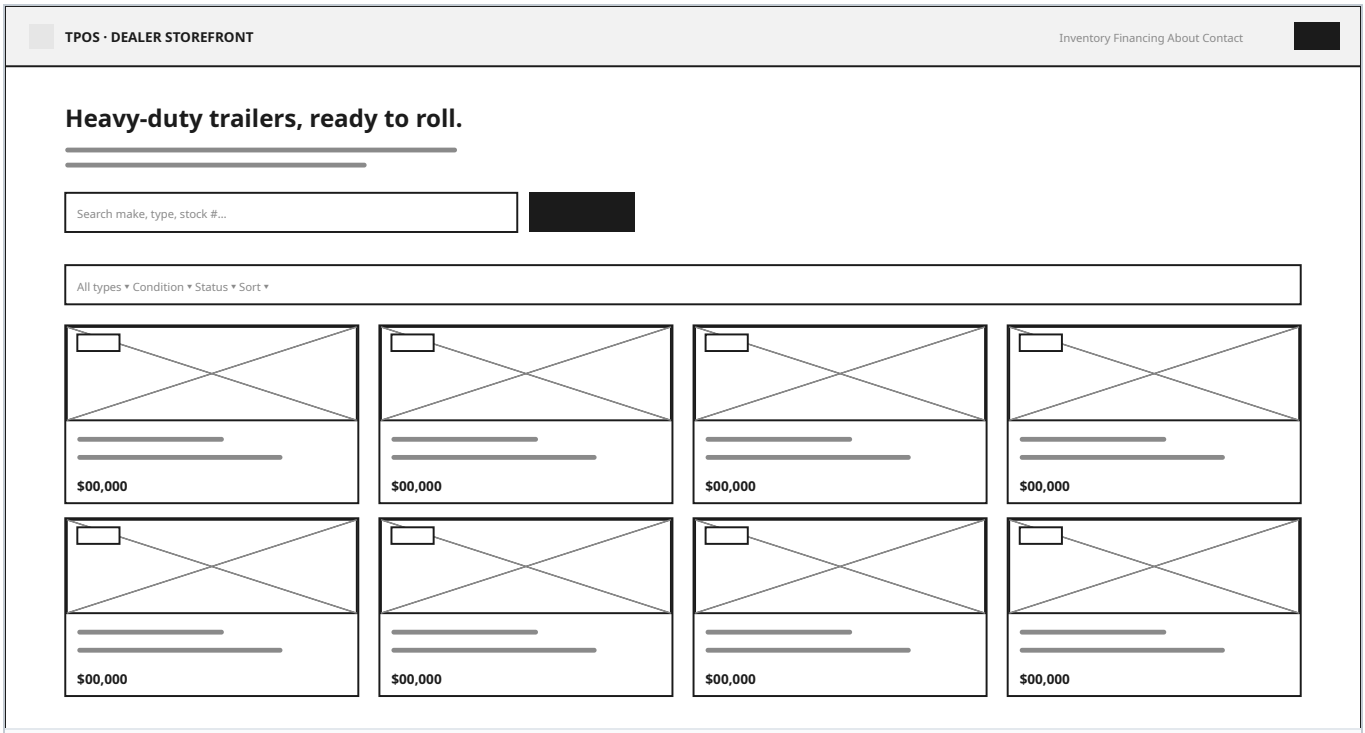
Wireframe — Platform Super-Admin primary workspace (layout & IA only).

7 · Public Storefront

Every tenant gets a branded, mobile-first public website auto-generated from its *available* inventory. It is the demand engine that feeds the Lead Desk.

Functional requirements

- Inventory grid with filter (type, condition, status), keyword search and sort (featured / price / year); each unit shows photos, key specs, price, location and a status badge.
- Deep-linkable, shareable detail pages per unit (full specs, photos, description) with status (Available / Pending / Sold).
- Lead capture: "Request a quote" and "Sell your trailer" forms create a **Lead** in the tenant's back office and notify staff; financing pre-qualification entry point.
- Informational pages (financing, about, contact) and tenant branding (logo, colors, phone, address).
- Units automatically appear/disappear as their status changes; optional syndication to external listing channels.



Public storefront — generated from available inventory; forms create leads in the back office.

8 · Cross-Cutting Capabilities

- **Global search** across units, leads, customers, deals, invoices and documents, scoped to the tenant and the user's permissions.

- **Notifications** (in-app + email/SMS) for new leads, approvals needed, overdue AR, delivery updates and title actions.
- **Activity log / audit trail** on every entity: status changes, price changes, discounts, payments, document signatures — who, when, why.
- **Comments & @mentions** on units, deals and work orders so roles hand off context.
- **Tasks & follow-ups** assignable to users, surfaced on each role's dashboard.
- **Document storage & e-signature** for titles, bills of sale, financing and delivery receipts.

9 · Reporting & KPIs

All dashboards derive from the shared record so figures reconcile across roles. Core KPIs the system must compute:

Domain	KPIs
Sales	Pipeline value, weighted forecast, win rate, average deal, units sold (period), revenue, gross profit, sales-cycle length
Inventory	Units in stock by status, inventory value & cost, average days on lot, aged units (90+), turn rate, by-location counts
Pricing	Average margin %, units below floor, margin by type, reprice impact
Marketing	Leads by source, cost per lead, first-response time, lead→quote→won conversion
Service	Open/in-progress/waiting work orders, recon cost & cycle time, hours utilization
Money	AR aging, AP due, cash position, gross margin, monthly P&L
Platform (operator)	MRR / ARR, net-new MRR, churn, trials & conversions, seats & units per tenant, usage, uptime

10 · Roles & Permissions Matrix

Access is role-based and tenant-scoped. A user may hold multiple roles (common in small dealers). Sample matrix (✓ = full, ◑ = own records, · = read-only, blank = none):

Capability	Owner	Lead	Sales	S.Mgr	Price	Inv	Svc	Office	Books	Disp	Cust
View inventory	✓	·	·	·	✓	✓	·	·	·	·	store
Edit price / floor	✓			◑	✓	·					
Work leads	·	✓	◑	✓							
Advance deals	·		◑	✓				·	·	·	own
Approve discount	✓			✓	◑						

Work orders	·					✓	✓				
Titles / documents	·		·					✓	·	·	own
Invoices / payments	·							·	✓		own
Schedule delivery	·		·			·		·		✓	·
Manage users / billing	✓										

The Platform Super-Admin operates above all tenants and is not on this per-tenant matrix.

11 · External Integrations

Integration	Purpose
Financing / lenders	Soft-pull pre-qualification, application submission, decision status into Deal & Customer portal
Listing syndication	Push available units to TruckPaper, Facebook Marketplace, Google; pull leads back
Title / DMV & e-sign	Title status, lien release, electronic signing of titles & bills of sale
Payments	Deposits, payoffs, ACH/wire/card; reconcile to invoices
Freight / carriers	Freight rate quotes, dispatch, in-transit tracking
Accounting export	Sync to QuickBooks / Xero (invoices, payments, journal entries)
Email & SMS	Lead notifications, customer comms, delivery/title updates
Platform billing	Subscription billing & metering for the SaaS operator (hosted mode)

12 · Non-Functional Requirements

Area	Requirement
Multi-tenancy	Strict per-tenant data isolation enforced at the data layer; no cross-tenant access possible by any query path.
Security	Authentication with sessions/SSO; role-based authorization on every action; encrypted secrets; audit log of sensitive actions; PII & financing data protected.
Availability	Hosted SLA target 99.9%; storefront cached/CDN-served; graceful degradation if back-office services are down.
Performance	Dashboards load < 2 s on typical tenant volumes; storefront fast on mobile; search responsive.
Scalability	Hundreds of tenants, tens of thousands of units & leads per tenant, without per-tenant degradation.
Portability	Same codebase deploys hosted (multi-tenant) and self-host (single-tenant) via configuration/license; no operator-only assumptions in tenant features.
Data & backups	Per-tenant export of all data; regular backups; tenant offboarding returns/erases data.
Observability	Logs, metrics, error tracking; per-tenant usage metering for billing; uptime & API latency dashboards (operator).
Accessibility & responsive	Internal tools usable on tablet; storefront fully responsive; WCAG-AA color/contrast for storefront.
Auditability	Immutable history of status/price/discount/payment/document events with actor, timestamp and reason.

13 · Acceptance Criteria

The rebuild is "done" when the following end-to-end scenarios pass for a tenant, with figures reconciling across roles:

1. **Acquire→list.** Inventory Manager adds an incoming unit; Service completes intake & recon; Pricing sets list/floor; status flips to *available* and the unit appears on the storefront and syndication.
2. **Lead→quote.** A storefront "Request a quote" creates a Lead in the Lead Desk within seconds, routes to a salesperson, who builds a Quote tied to the unit.
3. **Negotiate→approve.** A below-floor offer requires Sales Manager approval, which is logged; the Deal advances to financing.
4. **Finance→title→deliver.** Customer applies for financing in the portal and sees status; Office tracks title to "in hand"; Dispatch schedules and tracks delivery to "delivered."

5. **Book.** Bookkeeper issues the invoice, records payment, and the sale appears in revenue/gross KPIs that match the Owner dashboard.
6. **Operate (hosted).** Super-Admin provisions a new tenant, sees it billed, and watches MRR/usage update.

14 · Glossary

Floor price	The lowest price a unit may be sold at without manager approval; protects margin.
Days on lot	Days since acquisition; drives aging & reprice decisions.
Recon	Reconditioning — the service work to make an acquired unit retail-ready.
Pipeline (value)	Sum of open-deal values; "weighted" multiplies each by win probability.
AR / AP	Accounts receivable (money owed to the dealer) / payable (money the dealer owes).
Syndication	Auto-publishing inventory to external listing channels.
Tenant	One dealership's isolated instance of the platform.
MRR / ARR	Monthly / annual recurring revenue (operator metric).

Appendix A · Data Dictionary & Relational Schema

Recommended target: **PostgreSQL** (hosted / multi-tenant) with **Row-Level Security**; **SQLite** for single-tenant self-host installs behind the same data-access layer. Conventions applied to every tenant-scoped table:

- Primary key `id uuid`; audit columns `created_at`, `updated_at timestamptz`; optional `deleted_at` for soft delete.
- `tenant_id uuid NOT NULL → tenants(id)` on every business table, enforced by an RLS policy `USING (tenant_id = current_setting('app.tenant_id')::uuid)` — no query path can cross tenants. (Self-host keeps the column for parity and drops the policy.)
- Money as `numeric(12,2)`; flexible attributes as `jsonb`; enums as Postgres enum types / CHECK constraints (see below).
- Every mutation that matters (status, price, discount, payment, signature) writes a row to `activities` (immutable audit log).

Table	Key columns (type)	Relationships & indexes
tenants	name, plan(enum), status(enum), domain, branding(jsonb), billing_ref, lots(jsonb)	root of isolation; platform-managed
users	name, email, password_hash, roles(text[]), lot_scope(text[]), active(bool), last_login	→ tenants; RBAC subject; uniq(tenant_id,email)
units (inventory — the spine)	stock_no, type, year, make, length, axles, capacity, condition, status(enum), lot_id, cost, list_price, floor_price, acquired_at, sold_at, views, lead_count, specs(jsonb)	uniq(tenant_id,stock_no); idx(tenant_id,status); → lots
unit_photos	unit_id, url, sort, is_cover(bool)	→ units (→ object storage)
leads	name, company, source, status(enum), value, phone, email, lost_reason	→ units (interest), → users (owner)
customers	name, company, phone, email, portal_user_id	buyer accounts; → users (portal)
deals	stage(enum), value, probability, financing_status	→ customers, → units, → users (salesperson)
quotes / **quote_lines**	quotes: total, valid_until, status · lines: kind(enum: price|prep|freight|tax|fee), label, amount	quotes → deals/units; lines → quotes
work_orders / **wo_tasks**	WO: type, status(enum), hours_est, hours_actual, parts_cost, opened_at, closed_at · task: label, done(bool), sort	WO → units, → users (tech); task → work_orders
invoices	kind(AR|AP), amount, status(enum), issued_at, due_at, note, vendor	→ customers / deals / units (nullable)
payments	amount, method, paid_at, reference	→ invoices; idempotent on reference
titles	status(enum), state, lien_holder, released_at	uniq(unit_id) → units

documents	kind, url, signed_at, signer, sign_request_id	→ deals / units; e-signature
deliveries	origin, destination, carrier, driver, status(enum), eta, freight_cost	→ deals, → units
activities (audit log)	entity_type, entity_id, actor_id, action, detail(jsonb), at	append-only; idx(tenant_id,entity_type,entity_id)
tasks / **notifications**	tasks: assignee_id, title, due_at, done, link(entity) · notif: user_id, kind, payload(jsonb), read_at	cross-cutting; → users

Enumerations

Enum	Values
unit.status	`incoming · recon · available · pending · sold`
lead.status	`new · working · quoted · won · lost`
deal.stage	`lead · quoted · negotiation · financing · delivery · won · lost`
work_order.status	`open · in_progress · waiting_parts · done`
invoice.status	`paid · due · overdue`
title.status	`in_hand · applied · pending · lien_release`
delivery.status	`scheduled · en_route · delivered`
tenant.plan / .status	`starter · growth · enterprise · license` / `active · trial · licensed · suspended`

Platform control plane (hosted mode only — separate schema, no tenant RLS)

Table	Key columns
subscriptions	tenant_id, plan, mrr, seats, started_at, status, billing_provider_ref
usage_metrics	tenant_id, period, units, leads, storefront_views, api_calls (for metering & billing)
support_tickets	tenant_id, subject, status, priority, opened_at
license_keys	key, owner, plan, seats, expires_at (validated by self-host installs)

Appendix B · API Reference

REST / JSON. **Namespaces:** `/api/v1/*` authenticated back-office (tenant from session/subdomain, every query tenant-scoped) · `/api/v1/portal/*` customer portal · `/api/public/*` storefront (no auth, tenant by host) · `/api/platform/*` operator control plane (super-admin, cross-tenant) · `/api/webhooks/*` inbound integrations. RBAC per endpoint (the §10 matrix); state-machine changes use explicit `…/transition` actions (validated + audited), not arbitrary PATCH; lists accept `?q=&status=&sort=&page=&limit=`; money/transition writes accept an `Idempotency-Key` header.

Auth & session

```
POST  /api/v1/auth/login            GET  /api/v1/auth/me
POST  /api/v1/auth/logout           POST /api/v1/auth/sso/callback
POST  /api/v1/auth/password/reset
```

Inventory & pricing

```
GET   /api/v1/units                 list/filter (type,status,location,aged)
POST  /api/v1/units                 intake / acquire
GET   /api/v1/units/:id             full record ("all lenses")   PATCH /api/v1/units/:id
POST  /api/v1/units/:id/transition  incoming→recon→available→pending→sold (guarded)
POST  /api/v1/units/:id/photos      GET /api/v1/units/:id/timeline    GET /api/v1/units/aging
PATCH /api/v1/units/:id/price       set list/floor (logged; may require approval)
GET   /api/v1/pricing/repricer      GET /api/v1/pricing/comps?unit=:id
GET   /api/v1/pricing/approvals     POST /api/v1/pricing/approvals/:id/decide
```

Leads, customers, deals, quotes

```
GET/POST /api/v1/leads              GET/PATCH /api/v1/leads/:id
POST  /api/v1/leads/:id/assign      POST /api/v1/leads/:id/convert  →deal
GET   /api/v1/sources               GET/POST /api/v1/campaigns
GET/POST /api/v1/customers          GET/PATCH /api/v1/customers/:id
GET/POST /api/v1/deals              GET/PATCH /api/v1/deals/:id
POST  /api/v1/deals/:id/stage       advance pipeline
POST  /api/v1/deals/:id/discount    POST /api/v1/deals/:id/approve  (manager RBAC)
GET/POST /api/v1/quotes             POST /api/v1/quotes/:id/send | /accept
GET   /api/v1/forecast              GET /api/v1/team/leaderboard
```

Service · accounting · titles · delivery

```
GET/POST /api/v1/workorders         POST /api/v1/workorders/:id/status
PATCH /api/v1/workorders/:id/tasks/:t   POST …/hours  …/parts  …/photos
GET/POST /api/v1/invoices           POST /api/v1/invoices/:id/payments (idempotent)
GET/POST /api/v1/bills              GET /api/v1/accounting/pnl?period=   POST /accounting/export
GET/PATCH /api/v1/titles/:unitId    GET/POST /api/v1/documents   POST /documents/:id/sign
GET/POST /api/v1/deliveries         POST /api/v1/deliveries/:id/status   POST /deliveries/quote
GET/POST /api/v1/appointments
```

Cross-cutting & tenant admin

```
GET   /api/v1/search?q=            GET /api/v1/notifications  POST …/:id/read
GET/POST /api/v1/tasks             GET /api/v1/activity       GET /api/v1/reports/:kpiSet
GET/POST /api/v1/users             PATCH /api/v1/users/:id (roles, lot scope)
GET/PATCH /api/v1/tenant/settings  branding · lots · config · billing
```

Storefront (public) & customer portal

```
GET   /api/public/inventory        available units (filter/search/sort)
GET   /api/public/inventory/:stock  detail (deep-linkable)
POST  /api/public/inquiry          creates a Lead + notifies   ← implemented in prototype
POST  /api/public/financing/prequalify
GET   /api/v1/portal/me            GET /api/v1/portal/deals
GET   /api/v1/portal/quotes/:id    POST …/accept
POST  /api/v1/portal/financing/apply  GET …/status
GET   /api/v1/portal/delivery/:id  GET /api/v1/portal/documents  POST …/:id/sign
GET/POST /api/v1/portal/saved
```

Platform control plane (hosted) & webhooks

```
GET/POST /api/platform/tenants     PATCH /api/platform/tenants/:id (plan/status)
GET   /api/platform/revenue        GET /api/platform/usage    GET /api/platform/billing
GET   /api/platform/support        GET /api/platform/health   POST /api/platform/license/verify
POST  /api/webhooks/financing      lender decision → Deal/portal
POST  /api/webhooks/payments       settled → reconcile invoice
POST  /api/webhooks/freight        carrier status → delivery
POST  /api/webhooks/syndication    listing-site lead → Lead
POST  /api/webhooks/title          DMV/title callback
```

Appendix C · Performance & Scaling

TPOS is **database-bound CRUD**: requests spend their time waiting on Postgres, not on application code. The performance and tenant-density wins therefore live in the **data layer and the connection pool**, not in an exotic runtime or premature sharding. This appendix is one optimization profile; apply it only to the degree the product needs.

Define the dimension first. "High performance" is not one thing, and the lever order changes with it. Decide which of these dominates before optimizing — over-investing in the wrong one is wasted complexity.

If the goal is…	Dominant levers, in order
p99 read latency (dashboards feel instant)	Materialized read models → cache hot reads → `tenant_id`-leading indexes → read replicas
Write / transaction throughput	Index discipline → prepared statements & batching → hash-partition hot tables → pool sizing
Tenants per server (density)	PgBouncer transaction pooling → edge-offload the storefront → shared-schema low footprint → async runtime

C.1 Indexing & query design — the foundation (and what makes RLS cheap)

- Every hot index **leads with `tenant_id`**, then the query's predicates: `(tenant_id,status)`, `(tenant_id,stock_no)`, `(tenant_id,due_at)`, `(tenant_id,stage)`, etc.
- **Keep Row-Level Security.** With the right composite index the planner pushes the policy predicate into the index scan, so RLS is cheap (reported order-of-magnitude: ~0.3 ms policy evaluation at 50 M rows / 10 K tenants; ~1–5% vs. app-level filtering). **The 100× penalty comes from a missing `tenant_id`-leading index, not from RLS itself.** Database-enforced isolation is worth the few percent — one cross-tenant leak is existential, and it protects you even when application code has a bug.
- Eliminate N+1s; use prepared statements; keep working sets index-resident.

C.2 Partitioning — secondary, and use HASH not list

- Partitioning is a lever only for the largest tables (e.g. `activities`, `units` at very high volume); the composite index above does most of the work first.
- With thousands of tenants, partition by **`HASH(tenant_id)` into a fixed number of partitions** (e.g. 16–64) for cache locality and scan pruning. **Do NOT list-partition one-per-tenant** — thousands of partitions blow up planner time and DDL. Time-range sub-partitioning suits the audit log.

C.3 Connection pooling — the real density lever

- Postgres is process-per-connection (~5 MB + context-switch per connection), so density comes from **PgBouncer in transaction-pooling mode** multiplexing many clients onto a small pool sized to roughly **3–5 backend connections per CPU core**.

- **Gotcha that interacts with RLS:** under transaction pooling a session-level `SET app.tenant_id` does not persist across statements. Set the tenant **per-transaction** with `SET LOCAL app.tenant_id = …` (or `set_config('app.tenant_id', …, true)`) so RLS still applies. Also avoid session-scoped prepared-statement / session-feature assumptions in transaction mode.

C.4 Read path — precompute the dashboards (CQRS-lite)

- The KPI dashboards (§9) aggregate across many rows. Precompute them into **materialized views / rollup tables** refreshed incrementally by the background worker, so a dashboard load is an O(1) indexed read, not a per-request fan-out join. This is the single biggest internal-latency win.
- Cache hot reads (Redis / Dragonfly or in-process) with event-driven invalidation on writes; route all dashboard/report traffic to **read replicas**.

C.5 Storefront at the edge

The public storefront's "available inventory" is the highest-volume, most cacheable traffic. Serve it from a **CDN / ISR edge**, invalidated when a unit's status changes. This removes the largest read load from the origin entirely and is often the cheapest way to raise effective tenants-per-server.

C.6 Runtime — a minor lever for DB-bound work

Framework benchmarks show per-request gaps **collapse once Postgres is in the loop** (compiled runtimes dominate only the compute-bound plaintext/JSON rounds). For DB-bound CRUD you will not get throughput from switching language — choose the runtime for its **concurrency model and your team**, not per-query speed. Where it touches density: an async / light-concurrency runtime (**Go, .NET, Node, or Elixir** for connection-heavy/real-time) holds more in-flight requests per box than a process-per-worker Python/Ruby setup. **Rust is not required** here and its productivity cost rarely pays back on this workload.

C.7 Scaling ladder — earn each step

1. Vertical scaling + PgBouncer (handles a large share of workloads on its own).
2. Add read replicas for the read/report load.
3. Hash-partition the few largest tables.
4. **Only** once a single primary is genuinely exhausted (commonly hundreds of GB+), shard with **Citus** co-located by `tenant_id`. Keep the schema partition-ready (`tenant_id` on every table) so this migration is cheap — but **defer it**; vanilla Postgres + replicas goes much further than teams expect.

Caveat — treat the numbers as order-of-magnitude. The RLS timing figures come from a single practitioner benchmark and the single-node size thresholds are partly vendor framing. Use them for intuition; **benchmark on your own schema, queries and data** before treating any figure as a constant. Sources: practitioner RLS-at-scale benchmarks, PgBouncer pooling guides (connection-cost & per-core sizing), single-node-Postgres scaling write-ups, and the TechEmpower framework benchmarks (database-query rounds).

Appendix D · Architecture Decisions & Roads Not Taken

Every significant fork weighed for the rebuild — the options considered, what was chosen, what was rejected and why. Decisions are advisory; the build team owns the final call. Several were **revised mid-review as evidence came in** — those are flagged (REVISED) and recorded honestly rather than quietly replaced. Bracketed numbers reference the sources in §D.2.

#	Decision	Options considered	Chosen	Roads not taken — reasoning
1	Prototype persistence	Flat JSON file · real database	JSON file + in-browser mock	The prototype is a UI/IA demo; only the storefront lead endpoint is live. A real DB was out of scope for a front-end mock.
2	Production database	PostgreSQL · document/NoSQL · MySQL · SQLite-only	**PostgreSQL** (hosted) + SQLite per self-host tenant	NoSQL rejected — the domain is intensely relational (one-record-many-lenses = foreign keys), needs transactions (booking a sale) and joins. MySQL viable; Postgres preferred for RLS, jsonb, partitioning, extensions. SQLite-only rejected for hosted (concurrency) but ideal for a single self-host tenant. [1]
3	Multi-tenant isolation	Shared schema + RLS · schema-per-tenant · database-per-tenant · app-level filtering only	**Shared schema + RLS**, `tenant_id` on every table	DB-per-tenant rejected — poor density, heavy ops. Schema-per-tenant rejected — migration/ops complexity at scale. App-filtering-only rejected — one forgotten `WHERE` leaks data; isolation belongs in the database. RLS gives DB-enforced safety at ~1–5% when indexed. [1][2][3][4]
4	Service topology	Modular monolith · microservices	**Modular monolith** (one deployable)	Microservices rejected — the self-host requirement wants one artifact, the team is small, and network hops hurt DB-bound latency. Extract a service later only if a piece proves it needs it.
5	App framework (pragmatic profile)	Django/DRF · Rails · NestJS+Next.js · Go · .NET · Elixir	**Django/DRF** (lead) or **NestJS+Next.js** (JS-first)	Go/.NET/Rust rejected as the default — more boilerplate for CRUD/admin and no free admin plane. Chosen for batteries-included CRUD, migrations, RBAC, Postgres-first tooling, and clean self-host packaging.
6	Media storage	S3-compatible object storage (MinIO self-host) · blobs in Postgres	**S3-compatible** object storage	Blobs-in-DB rejected — bloats the database and slows backups/replication; photos & signed documents belong in object storage.
7	Storefront rendering	Edge/CDN (ISR) · in-app SSR · SPA	**SSR + edge-cache** available inventory	Pure SPA rejected for the public site — SEO & first-paint matter. Internal

				dashboards stay SPA/API (or SSR-with-enhancement).
8	Runtime under *performance-primary* **REVISED**	Rust · Go · .NET · Elixir · keep pragmatic	Async/light-concurrency (**Go/.NET/Elixir**); **Rust not required**	Initially proposed Rust for raw throughput. Evidence: framework gaps **collapse once Postgres is in the loop**; Rust leads only compute-bound rounds. For DB-bound CRUD the runtime is a minor lever — choose for concurrency model + team, not per-query speed. [9]
9	RLS under *performance-primary* **REVISED**	Keep RLS (indexed) · move isolation to app/partitioning for speed	**Keep RLS** + `tenant_id`-leading composite indexes	Initially suggested moving isolation out of RLS for speed. Evidence: RLS ~0.3 ms at 50 M rows / 10 K tenants *with the right composite index*; the 100× penalty is the **missing index**, not RLS. Keep it — the safety is worth ~1–5%. [2][3]
10	Horizontal sharding **REVISED**	Citus early (ease later migration) · single-node + replicas, defer	**Single node + replicas; defer Citus**, keep schema shard-ready	Initially put Citus "on the table." Evidence: vanilla Postgres + pooling handles most workloads under ~500 GB; replicas/partitioning extend into the terabytes before sharding is warranted. Defer until one primary is genuinely exhausted. [5][6]
11	Partitioning	HASH(tenant_id) fixed N · list/partition-per-tenant	**HASH(tenant_id)** into 16–64 partitions (large tables only)	Partition-per-tenant rejected — thousands of tenants blow up planner time & DDL. Partitioning is a *secondary* lever after the composite index. [6]
12	Connection management	PgBouncer transaction pooling · session pooling · none	**PgBouncer (transaction mode)** ~3–5/core + `SET LOCAL` tenant	No/session pooling rejected for density — process-per-connection (~5 MB each). Transaction mode requires `SET LOCAL app.tenant_id` per-transaction so the RLS variable survives (session `SET` silently breaks under it). [7][8]
13	Data access (perf paths)	Heavy ORM · thin typed SQL	**Thin typed SQL** (sqlx/sqlc/Dapper/Ecto) on hot paths	Heavy ORM rejected on hot paths — per-query overhead and hidden N+1s. ORM is fine for cold CRUD.
14	Dashboard read path	Per-request aggregation · materialized read models	**Materialized views / rollups** (CQRS-lite) + replicas + cache	Per-request big-join aggregation rejected for dashboards — precompute and serve an O(1) indexed read.

D.1 What we changed our minds about (and why)

Three decisions from the first "performance-first" pass were corrected once benchmark evidence was reviewed — recorded here so the reasoning is auditable, not lost:

- **#8 — Rust is not the lever.** For DB-bound CRUD, runtime choice barely moves throughput; the language win is real only on compute-bound paths. [9]

- **#9 — Don't drop RLS for speed.** The cost is the missing `tenant_id`-leading index, not the policy; keep DB-enforced isolation. [2][3]
- **#10 — Don't shard early.** A single Postgres primary + replicas + partitioning goes much further than expected; defer Citus. [5][6]

D.2 References & evidence quality

Sources consulted, with their nature flagged. **Several are vendor or single-practitioner sources; all numeric figures are order-of-magnitude guidance — benchmark on your own schema, queries and data before treating any as a constant.**

#	Source	Nature	Informs
1	PostgreSQL docs — Row Security Policies	Official documentation	RLS canonical reference (#2,#3)
2	RLS-at-scale benchmark (DEV Community)	Single-practitioner benchmark	~0.3 ms at 50 M/10 K with `tenant_id`-leading index; missing index = 100× (#3,#9)
3	RLS overhead write-up (Wellally)	Practitioner	~1–5% vs app-level filtering; planner push-down (#3,#9)
4	Tenant-isolation rationale (OneUptime); Crunchy Data RLS-for-tenants	Vendor / practitioner	DB-enforced isolation protects against app bugs (#3)
5	Single-node Postgres scaling (Velodb)	Practitioner / vendor	Vertical + pooling under ~500 GB; OpenAI on Postgres; replicas+partitioning to TB (#10)
6	Citus multi-tenant sharding docs & blog	Vendor framing	Shard by `tenant_id`, co-location; "~100 GB" threshold (#10,#11)
7	Connection scaling (PlanetScale); sizing (Akamai)	Vendor	Process-per-connection cost; PgBouncer multiplexing; ~3–5 conns/core (#12)
8	"PgBouncer … fraught with peril" (JP Camara)	Practitioner deep-dive	Transaction-pooling gotchas; `SET LOCAL` / session-feature caveats (#12)
9	TechEmpower Framework Benchmarks	Independent benchmark	DB-query rounds collapse framework gaps; Rust leads only plaintext/JSON (#8)